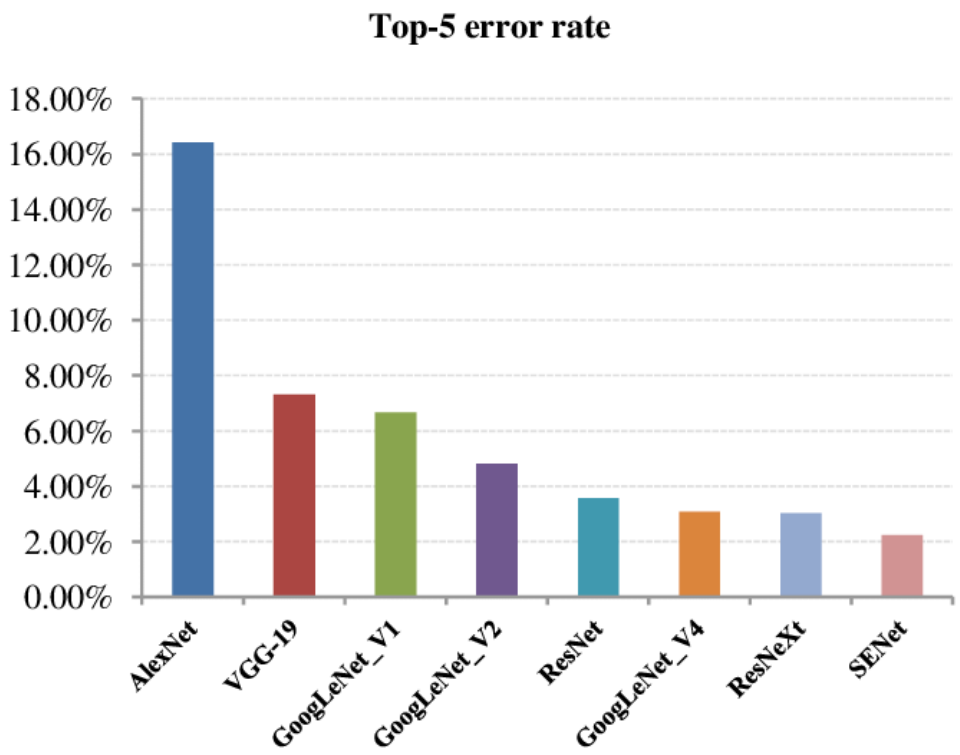
CNN卷積神經網路

目錄

- 經典模型比較
- GoogLeNet
  - EX1_inception v1

比較



模型名	LeNet	AlexNet	VGG	GoogLeNet	ResNet
發表	1998	2012	2014	2014	2015
層數	5	8	19	22	152
Top-5錯誤	沒測過	16.4%	7.3%	6.7%	3.57%

GoogLeNet特色與影響

- 2014 年 ImageNet比賽(ILSVRC14)第一名
- 提出了Inception架構(多尺度的卷積處理)
- 把Ensemble的思想用到一個極致
- 提出1x1的卷積
- 據說GoogLeNet的L要大寫是為了致敬LeNet

GoogLeNet介紹

References

[1] Know your meme: We need to go deeper. http://knowyourmeme.com/memes/we-need-to-go-deeper. Accessed: 2014-09-15.

又稱Inception Net

GoogLeNet架構

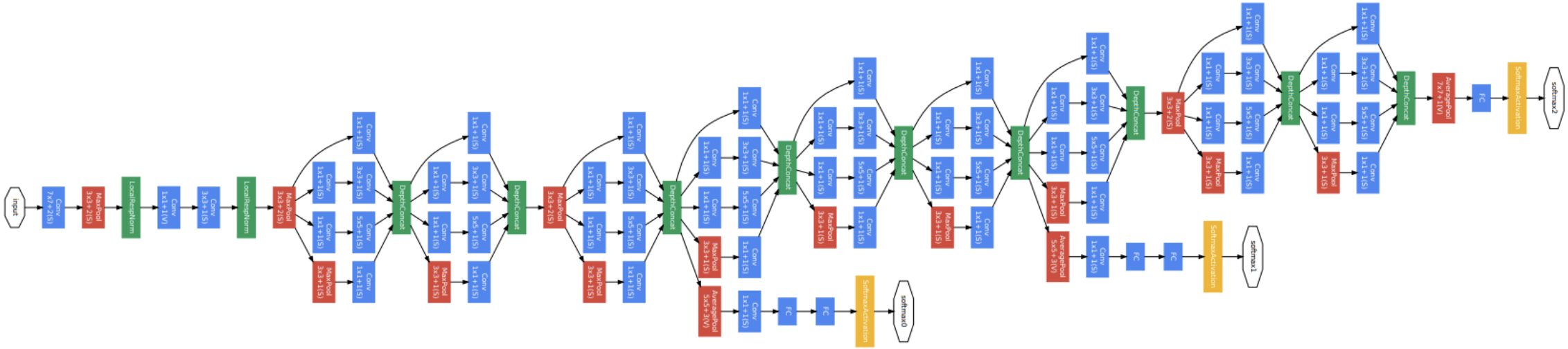
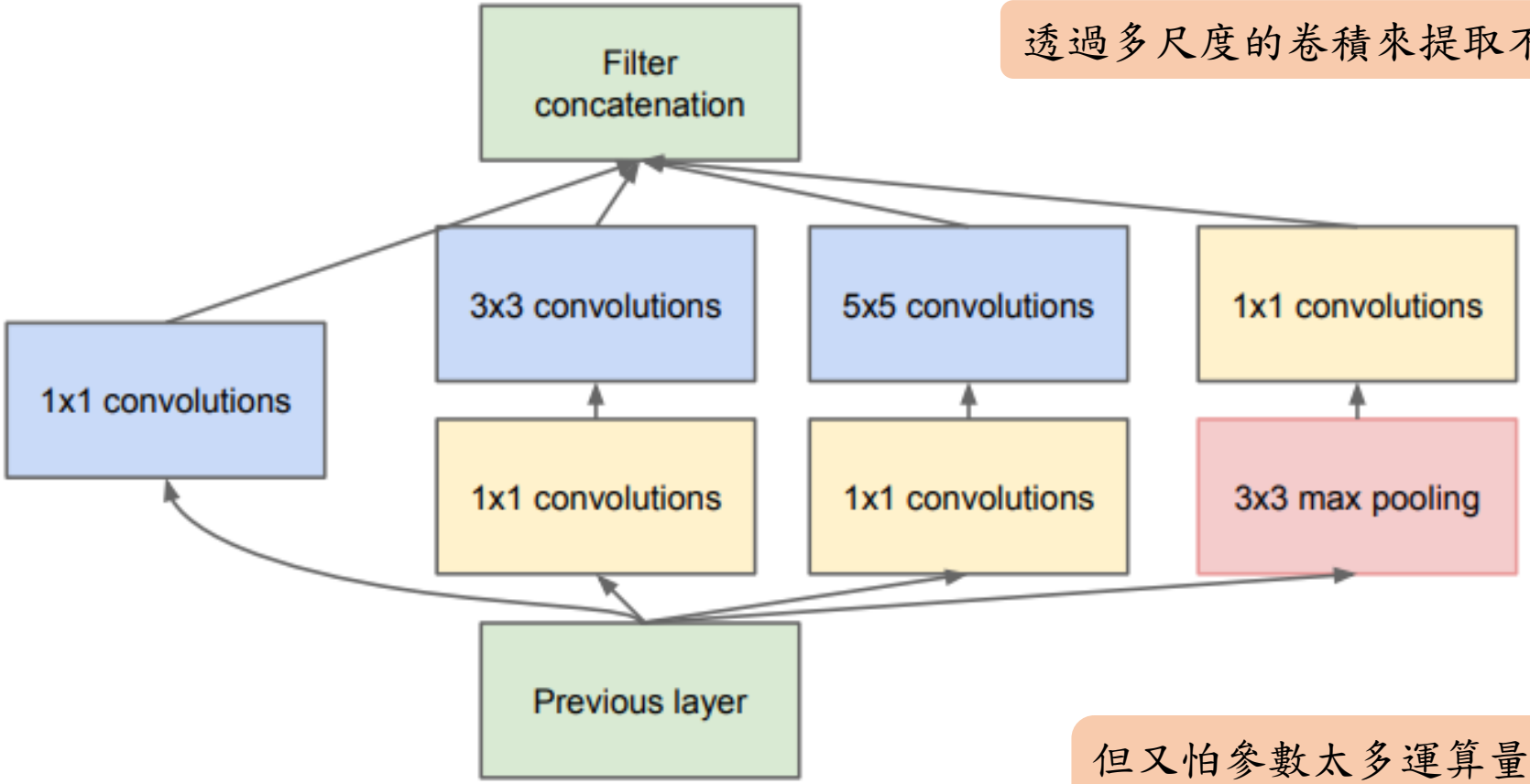


Figure 3: GoogLeNet network with all the bells and whistles

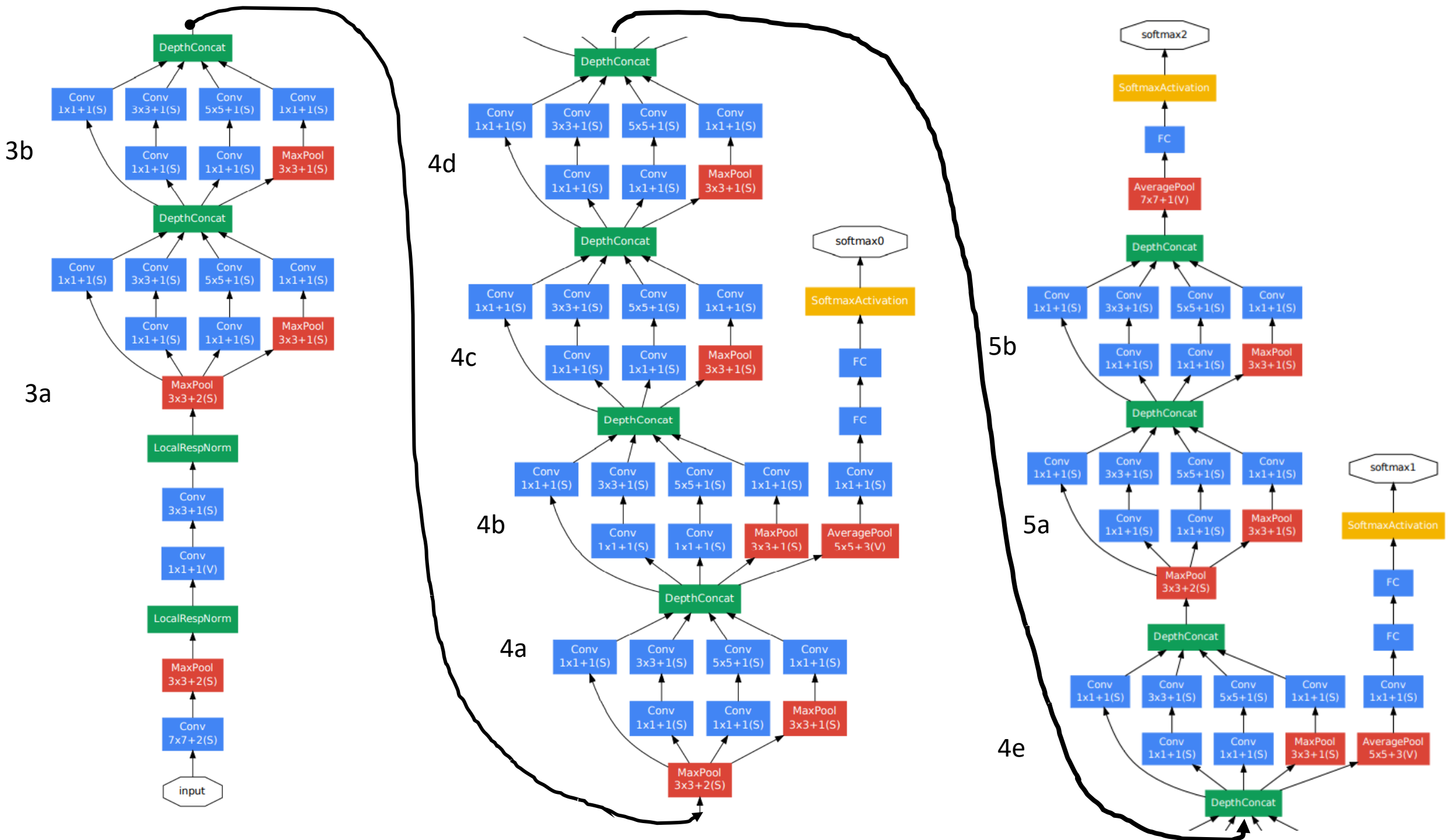
GoogLeNet : https://arxiv.org/pdf/1409.4842.pdf

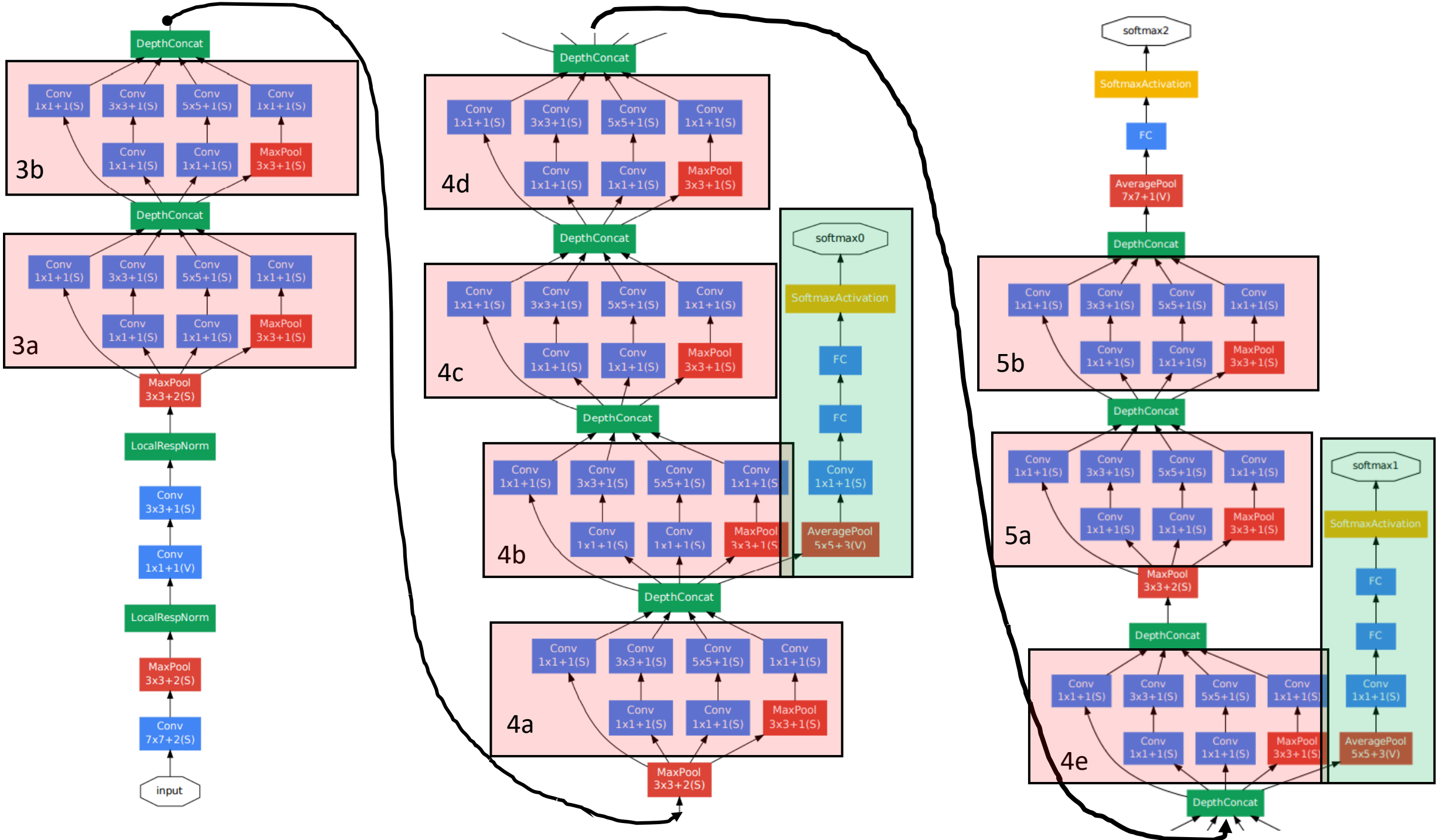
Inception



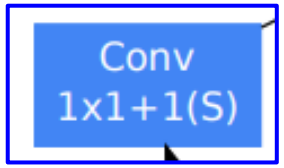
透過多尺度的卷積來提取不同的特徵

但又怕參數太多運算量爆表，
所以加入1x1卷積降維

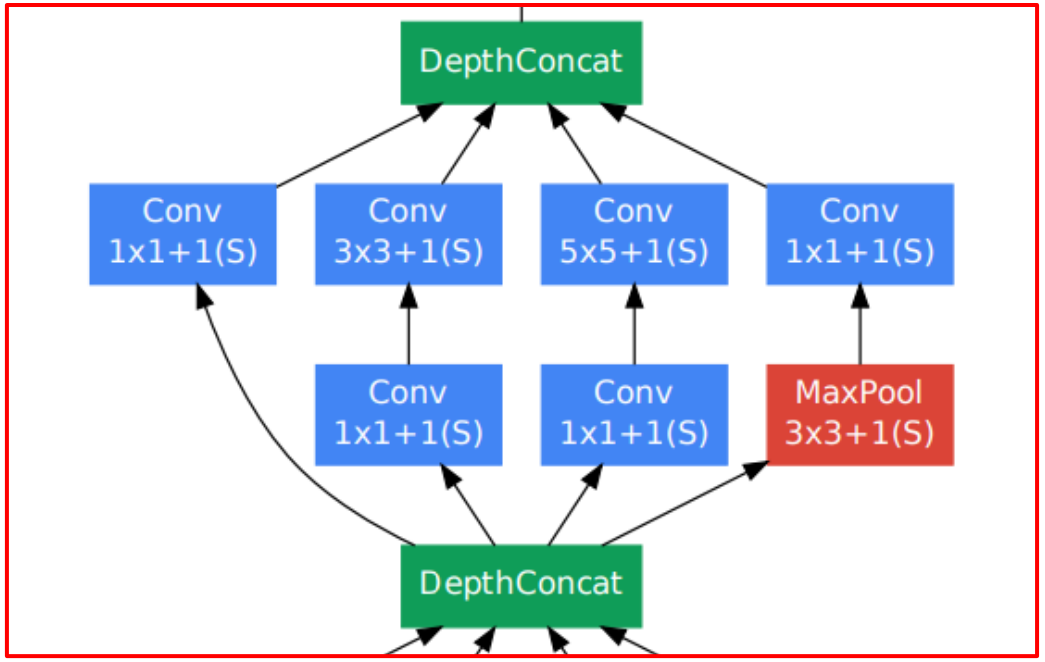

3b
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
3a
MaxPool 3x3+2(S)
LocalRespNorm
Conv 3x3+1(S)
Conv 1x1+1(V)
LocalRespNorm
MaxPool 3x3+2(S)
Conv 7x7+2(S)
input
4d
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
4c
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
4b
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
4a
MaxPool 3x3+2(S)
softmax0
SoftmaxActivation
FC
FC
Conv 1x1+1(S)
AveragePool 5x5+3(V)
5b
softmax2
SoftmaxActivation
FC
AveragePool 7x7+1(V)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
5a
MaxPool 3x3+2(S)
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
AveragePool 5x5+3(V)
4e
DepthConcat
softmax1
SoftmaxActivation
FC
FC
Conv 1x1+1(S)


input
Conv 7x7+2(S)
MaxPool 3x3+2(S)
Conv 1x1+1(V)
Conv 3x3+1(S)
LocalRespNorm
LocalRespNorm
MaxPool 3x3+2(S)
3a
DepthConcat
Conv 1x1+1(S)
Conv 3x3+1(S)
Conv 5x5+1(S)
Conv 1x1+1(S)
MaxPool 3x3+1(S)
3b
DepthConcat
4a
4b
4c
4d
4e
5a
5b
softmax0
softmax1
softmax2
AveragePool 5x5+3(V)
AveragePool 7x7+1(V)
FC
SoftmaxActivation

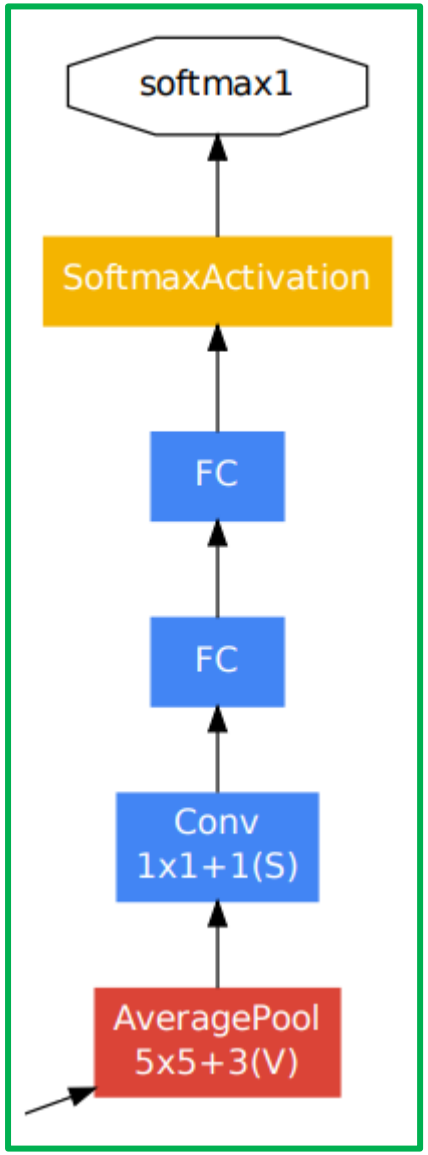
① Conv block



② Inception block



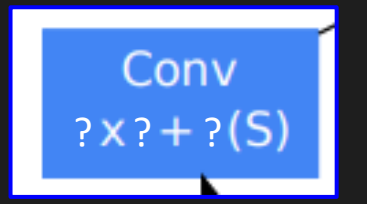
③ auxiliary classifiers



EX1_GoogLeNet(①Conv block)

```python
class conv_block(nn.Module):
    def __init__(self, in_channels, out_channels, **kwargs):
        super(conv_block, self).__init__()
        self.conv = nn.Conv2d(in_channels, out_channels, **kwargs)
        self.batchnorm = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.relu(self.batchnorm(self.conv(x)))
```



EX1_GoogLeNet(②Inception block)

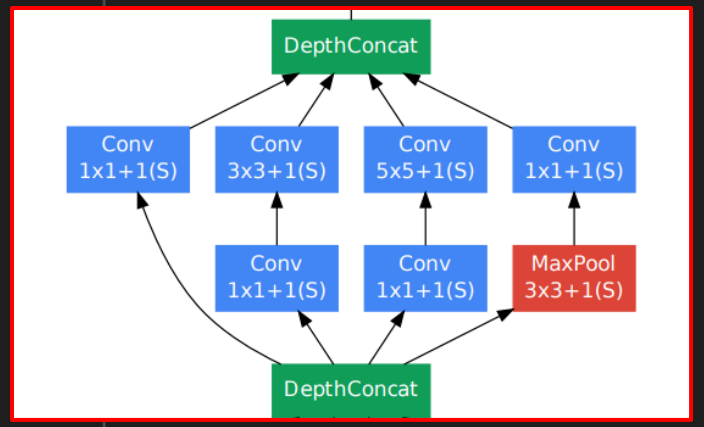
這些都還只是變數，(red表示reduction)

```python
class Inception_block(nn.Module):
    def __init__(self, in_channels, out_1x1, red_3x3, out_3x3, red_5x5, out_5x5, out_1x1pool):
        super(Inception_block, self).__init__()
        self.branch1 = conv_block(in_channels, out_1x1, kernel_size=(1, 1))

        self.branch2 = nn.Sequential(
                conv_block(in_channels, red_3x3, kernel_size=(1, 1)),
                conv_block(red_3x3, out_3x3, kernel_size=(3, 3), padding=(1, 1)))

        self.branch3 = nn.Sequential(
                conv_block(in_channels, red_5x5, kernel_size=(1, 1)),
                conv_block(red_5x5, out_5x5, kernel_size=(5, 5), padding=(2, 2)))

        self.branch4 = nn.Sequential(
                nn.MaxPool2d(kernel_size=(3, 3), stride=(1, 1), padding=(1, 1)),
                conv_block(in_channels, out_1x1pool, kernel_size=(1, 1))
                )
    def forward(self, x):
        return torch.cat([self.branch1(x), self.branch2(x), self.branch3(x), self.branch4(x)], 1)
```

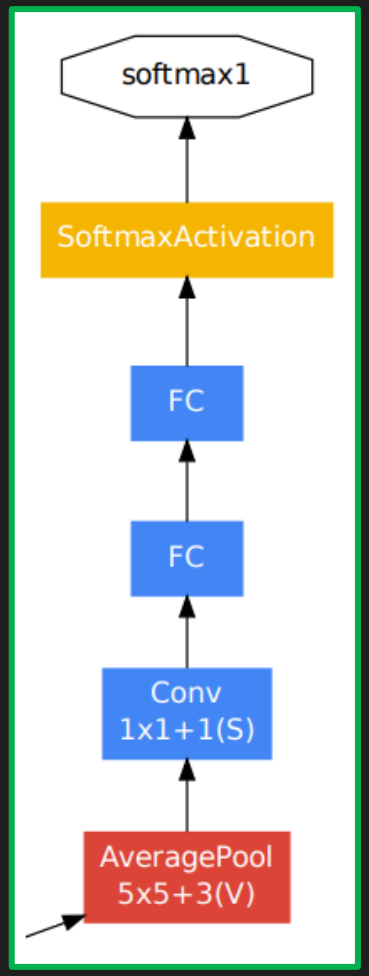


四個分支

把4個分branch的結果疊加起來

EX1_GoogLeNet(③InceptionAux)

```python
class InceptionAux(nn.Module):
    def __init__(self, in_channels, num_classes):
        super(InceptionAux, self).__init__()
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(p=0.7)  隨機丟到70%的神經元
        self.pool = nn.AvgPool2d(kernel_size=5, stride=3)
        self.conv = conv_block(in_channels, 128, kernel_size=1)
        self.fc1 = nn.Linear(2048, 1024)
        self.fc2 = nn.Linear(1024, num_classes)
    def forward(self, x):
        x = self.pool(x)
        x = self.conv(x)
        x = x.reshape(x.shape[0], -1)
        x = self.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

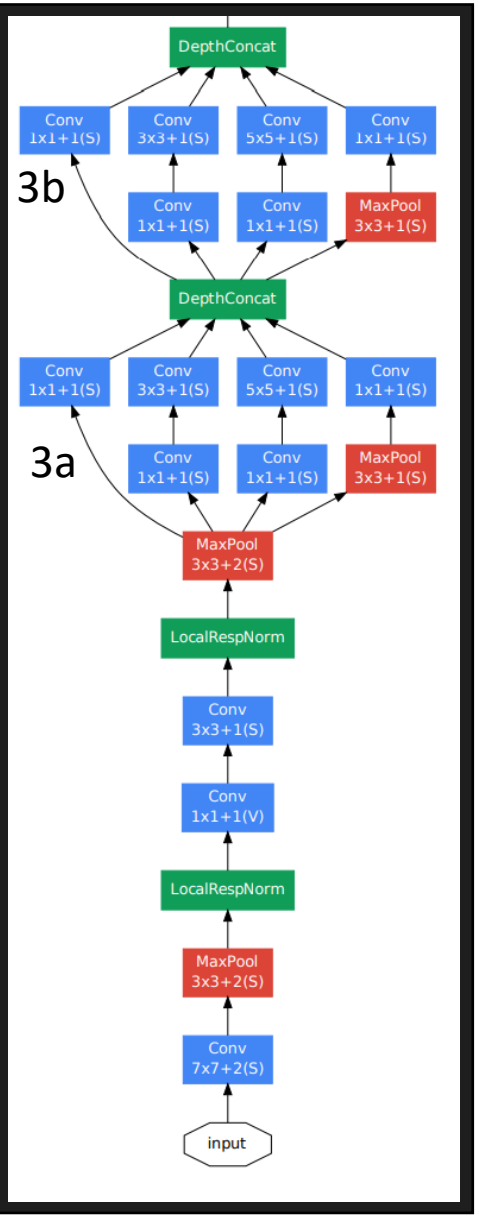


EX1_GoogLeNet(主體)

```python
class GoogLeNet(nn.Module):
    def __init__(self, aux_logits=True, num_classes=2):
        super(GoogLeNet, self).__init__()
        assert aux_logits == True or aux_logits == False
        self.aux_logits = aux_logits

        self.conv1 = conv_block(in_channels=3,out_channels=64, kernel_size=(7, 7),
                                stride=(2, 2),padding=(3, 3))
        self.maxpool1 = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)

        self.conv2 = conv_block(64, 192, kernel_size=3, stride=1, padding=1)
        self.maxpool2 = nn.MaxPool2d(kernel_size=3, stride=2, padding=1)
                                in_channels, out_1x1, red_3x3, out_3x3, red_5x5, out_5x5, out_1x1pool
        self.inception3a = Inception_block(192, 64, 96, 128, 16, 32, 32)
        self.inception3b = Inception_block(256, 128, 128, 192, 32, 96, 64)
        self.maxpool3 = nn.MaxPool2d(kernel_size=(3, 3), stride=2, padding=1)
```

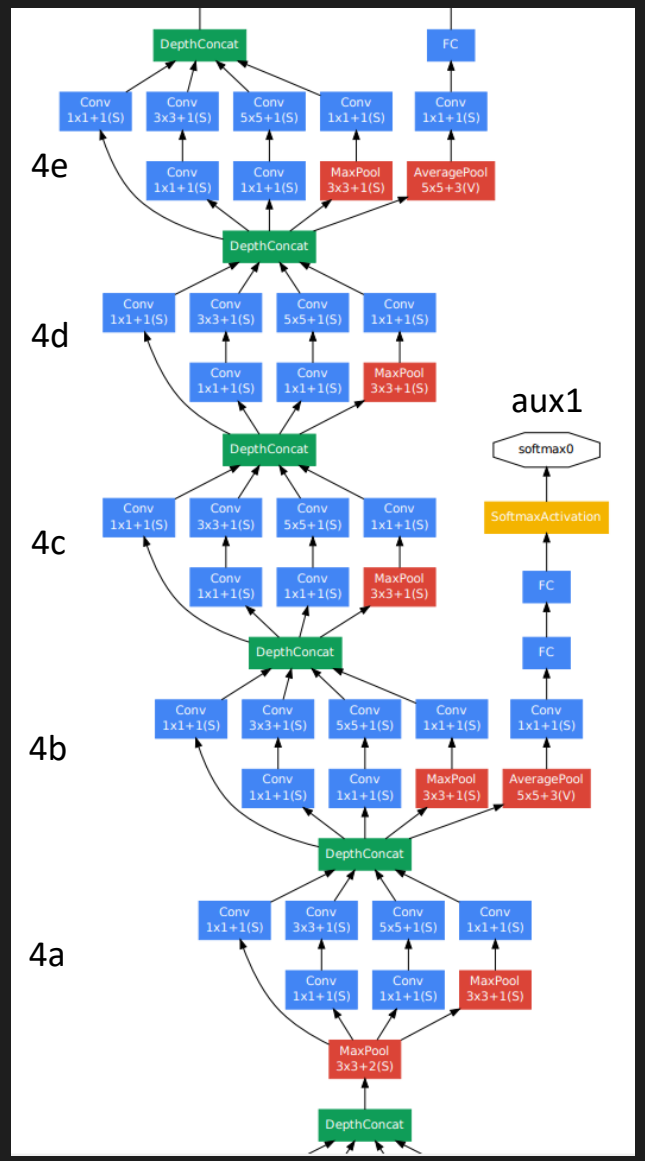


EX1_GoogLeNet(主體)

```
⋮

self.inception4a  =  Inception_block(480,  192,  96,  208,  16,  48,  64)
self.inception4b  =  Inception_block(512,  160,  112,  224,  24,  64,  64)
self.inception4c  =  Inception_block(512,  128,  128,  256,  24,  64,  64)
self.inception4d  =  Inception_block(512,  112,  144,  288,  32,  64,  64)
self.inception4e  =  Inception_block(528,  256,  160,  320,  32,  128,  128)
self.maxpool4  =  nn.MaxPool2d(kernel_size=3,  stride=2,  padding=1)

⋮
```



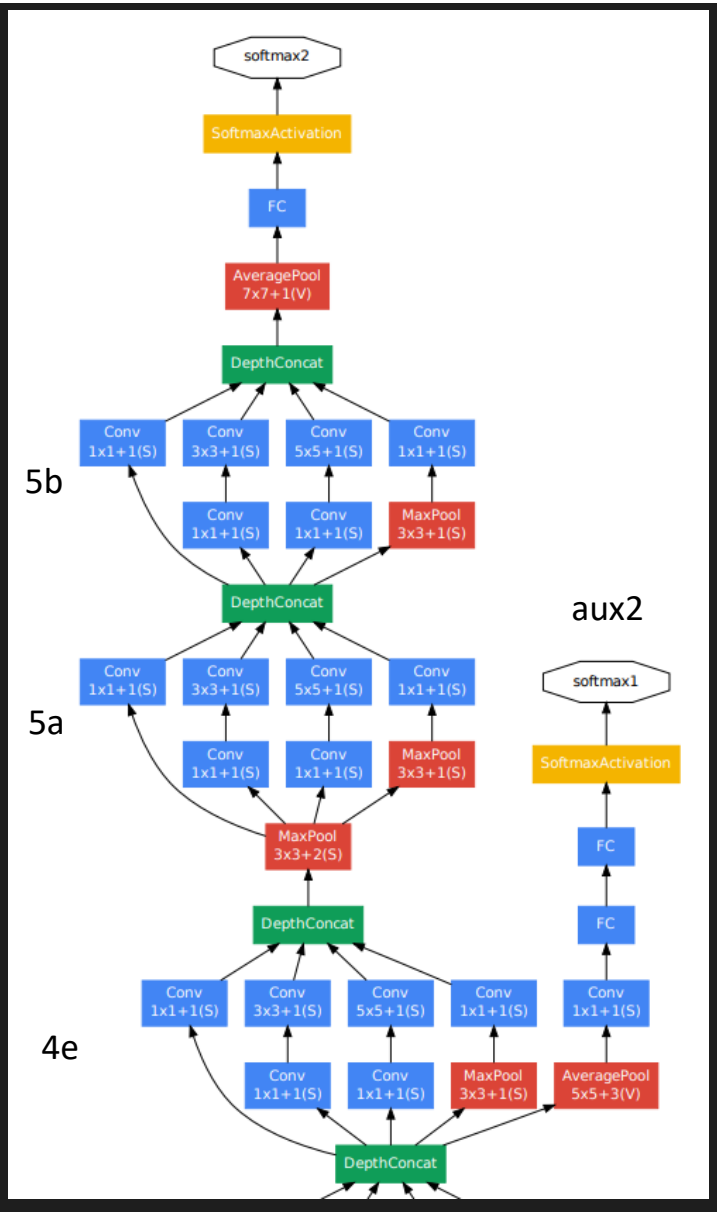
EX1_GoogLeNet(主體)

```
⋮

self.inception5a = Inception_block(832, 256, 160, 320, 32, 128, 128)
self.inception5b = Inception_block(832, 384, 192, 384, 48, 128, 128)

self.avgpool = nn.AvgPool2d(kernel_size=7, stride=1)
self.dropout = nn.Dropout(p=0.4)
self.fc1 = nn.Linear(1024, num_classes)

if self.aux_logits:
        self.aux1 = InceptionAux(512, num_classes)
        self.aux2 = InceptionAux(528, num_classes)
else:
        self.aux1 = self.aux2 = None
```



EX1_GoogLeNet(主體)

```python
def forward(self, x):
    x = self.conv1(x)
    x = self.maxpool1(x)
    x = self.conv2(x)
    x = self.maxpool2(x)
    x = self.inception3a(x)
    x = self.inception3b(x)
    x = self.maxpool3(x)
    x = self.inception4a(x)

    # Auxiliary Softmax classifier 1
    if self.aux_logits and self.training:
        aux1 = self.aux1(x)

    x = self.inception4b(x)
    x = self.inception4c(x)
    x = self.inception4d(x)

⋮
```

```python
⋮

    # Auxiliary Softmax classifier 2
    if self.aux_logits and self.training:
        aux2 = self.aux2(x)

    x = self.inception4e(x)
    x = self.maxpool4(x)
    x = self.inception5a(x)
    x = self.inception5b(x)
    x = self.avgpool(x)
    x = x.reshape(x.shape[0], -1)
    x = self.dropout(x)
    x = self.fc1(x)

    if self.aux_logits and self.training:
        return aux1, aux2, x
    else:
        return x
```

如果模型是model.train()
就會輸出分支

EX1_GoogLeNet(模型初始化)

```
model  =  GoogLeNet(aux_logits=True,  num_classes=10)
model  =  model.to('cuda')

optimizer  =  optim.Adam(model.parameters(),  lr=3e-4)
criterion  =  nn.CrossEntropyLoss()
```

EX1_GoogLeNet(開始訓練)

```
# Train Network
model.train()
for epoch in range(3):
    for batch_idx, (data, targets) in enumerate(tqdm(train_loader)):
        # Get data to cuda if possible
        data = data.to("cuda")
        targets = targets.to("cuda")

        # forward
        scores = model(data)

        loss0 = criterion(scores[0], targets)#
        loss1 = criterion(scores[1], targets)#
        loss2 = criterion(scores[2], targets)#
        loss = loss0*0.3+loss1*0.3+loss2#

        # backward
        optimizer.zero_grad()
        loss.backward()

        # gradient descent or adam step
        optimizer.step()
```

總loss為三個分支個別的損失，加權後的總合

EX1_GoogLeNet(驗證)

```
num_correct = 0
num_samples = 0
model.eval()

with torch.no_grad():
    for x, y in test_loader:
        x = x.to("cuda")
        y = y.to("cuda")

        scores = model(x)
        _, predictions = scores.max(1)
        num_correct += (predictions == y).sum()
        num_samples += predictions.size(0)

print(num_correct/num_samples)
```

因為不是訓練期，
所以輸出只有會後一個

```
tensor(0.8020, device='cuda:0')
```

練習(參考ipynb檔案)

繳交格式: 問題請直接使用文字回答即可，實作相關請附程式碼截圖證明還有文字說明

P1.Inception Block問題

1. 請問將input的channel，或是output的channel變成兩倍，對於我們的計算FLOPs會有甚麼影響?
2. 今天我in_channel=192, out_channel=128, 打算用3x3的捲積核。請問(1)直接使用3x3捲積與(2)先用1x1降到96後再3x3捲積輸出到128，兩者差多少參數? (參考公式，大概數值即可)

P2.GoogLeNet實作挑戰

1. 請設法調整訓練參數，讓訓練的過程中train acc與test acc均高於 45%
   Epoch [9/100] Loss: 3.2517 | Train Acc: 52.17%
   Val Loss: 1.8459 | Val Acc: 51.70%
   ---
2. 跑aux_logits=True, aux_logits=False,比較學習的趨勢與準確率
   注意:aux=false，training的程式會需要改兩個地方，請看註解處

參考

GoogleNet介紹: https://www.youtube.com/watch?v=agA3CDDIjZw&t=285s

GoogLeNet論文 : https://arxiv.org/pdf/1409.4842.pdf

Pytorch建inception-v1: https://www.youtube.com/watch?v=uQc4Fs7yx5I&t=675